# AulaGestión: aplicativo móvil para la gestión de inventario, recomendación y reserva de aulas en una institución educativa

María Alejandra Vallejo Zamora, Juan David Delgado Muñoz, Julián Andrés Ceballos Eraso y David Esteban Ortiz Ortiz

Programa de Ingeniería de Sistemas, Universidad Mariana

Desarrollo de Sistemas Web - Móvil

Sandro Favian Parra Pay

8 de noviembre de 2026

**Resumen**

Este documento presenta AulaGestión, un aplicativo móvil para Android que reúne en un solo lugar las aulas de una institución educativa, sus recursos, los horarios y las reservas. El proyecto parte de un problema frecuente: la información sobre espacios y equipos está dispersa, lo que provoca cruces de reservas y clases programadas en aulas que no tienen lo que la actividad necesita. La solución usa una arquitectura cliente-servidor: una aplicación Flutter consume una API REST en Laravel 13, protegida con tokens Sanctum y permisos por rol, que guarda los datos en MySQL 8. La API está publicada en Render con la base de datos en Aiven, y la aplicación se comunica con ella solo por HTTPS. El sistema atiende a tres perfiles (administrador, profesor y encargado de inventario) y cubre doce requerimientos funcionales. Entre ellos se destacan un recomendador basado en reglas, que descarta las aulas que no sirven y explica con una puntuación de 0 a 100 por qué sugiere cada una, y un sistema de alertas que avisa al profesor cuando el aula que reservó deja de estar disponible. Las reservas se confirman dentro de una transacción con bloqueo de fila para impedir superposiciones, y los cambios importantes quedan en una bitácora de auditoría. El trabajo se organizó con Scrum y se verificó con integración continua, 21 pruebas automatizadas del servidor, pruebas de interfaz y doce casos de aceptación. Se entrega la versión 1.2.1 con su instalador, la especificación OpenAPI, una colección de Postman y los manuales de instalación y uso.

*Palabras clave:* aplicación móvil, Flutter, Laravel, reserva de aulas, inventario, sistema de recomendación

## Tabla de contenido

Planteamiento del problema....5
Objetivos....6
Objetivo general....6
Objetivos específicos....6
Justificación....6
Alcance y limitaciones....7
Marco de referencia....7
Desarrollo móvil multiplataforma....7
Arquitectura REST....7
Autenticación basada en tokens....8
Persistencia relacional....8
Calidad, seguridad y protección de datos....8
Metodología....8
Requerimientos....9
Requerimientos funcionales....9
Requerimientos no funcionales....10
Roles y permisos....11
Reglas de negocio....12
Diseño de la solución....13
Arquitectura....13
Despliegue....14
Modelo de datos....14
Recomendador de aulas....16
Alertas de disponibilidad....18
Interfaz de usuario....19
Interfaz de programación....20
Implementación....21
Tecnologías....21
Organización del código....21
Seguridad....22
Pruebas....22
Pruebas automatizadas....23
Casos de aceptación....23
Pruebas de permisos y límites....24

Resultados.....24
Conclusiones.....26
Referencias.....27
Apéndice A. Manual de instalación y despliegue.....28
Apéndice B. Manual de usuario.....31
Apéndice C. Matriz de trazabilidad.....33
Apéndice D. Evidencias de funcionamiento.....34

# AulaGestión: aplicativo móvil para la gestión de inventario, recomendación y reserva de aulas en una institución educativa

En muchas instituciones educativas la asignación de aulas y el control de los equipos que contienen se resuelven con hojas de cálculo, cuadernos de préstamo o solicitudes verbales. Mientras el número de espacios es pequeño este método funciona, pero a medida que crecen los grupos, los horarios y los recursos tecnológicos aparecen cruces de reservas, aulas asignadas sin el videobeam o los computadores que la clase necesitaba y equipos dañados que nadie reporta a tiempo.

AulaGestión responde a esa situación con un aplicativo móvil que reúne en un solo lugar las aulas, su inventario, los horarios institucionales y las reservas. El profesor describe su actividad y el sistema le recomienda las aulas que realmente la soportan; el encargado de inventario mantiene al día el estado de cada recurso, y el administrador configura usuarios, espacios, jornadas y bloqueos.

El proyecto se desarrolló en la asignatura Desarrollo de Sistemas Web - Móvil del Programa de Ingeniería de Sistemas de la Universidad Mariana. Este documento describe el problema, los objetivos, la metodología, los requerimientos, el diseño, la implementación y las pruebas de la solución; los manuales de instalación y de usuario, junto con la matriz de trazabilidad, se incluyen como apéndices. El texto sigue las normas de la American Psychological Association (APA, 2020).

## Planteamiento del problema

La institución no cuenta con un registro unificado de sus espacios académicos ni de los recursos asignados a cada uno. La disponibilidad de un aula se consulta de forma manual, sin una regla que impida reservar la misma franja dos veces, y la información sobre equipos dañados, faltantes o en mantenimiento depende de reportes informales que no dejan historial.

Estas condiciones generan tres efectos concretos: clases que se superponen en el mismo espacio, actividades que se programan en aulas sin la capacidad o los recursos requeridos y decisiones de compra o mantenimiento tomadas sin datos confiables. De ahí surge la pregunta

que orientó el proyecto: ¿cómo puede un aplicativo móvil centralizar el inventario y la disponibilidad de las aulas para que los profesores reserven el espacio adecuado sin conflictos y la institución conserve trazabilidad de sus recursos?

## Objetivos

### Objetivo general

Desarrollar un aplicativo móvil para Android, apoyado en una API REST, que permita gestionar el inventario de las aulas de una institución educativa, recomendar el espacio más adecuado para cada actividad y registrar reservas sin superposiciones.

### Objetivos específicos

1. Levantar y consolidar los requerimientos funcionales y no funcionales con los criterios de aceptación del equipo.
2. Diseñar una arquitectura cliente-servidor y un modelo de datos que separen la interfaz de las reglas de negocio.
3. Implementar la autenticación, la autorización por roles y los módulos de usuarios, aulas, inventario, horarios, reservas, notificaciones y reportes.
4. Construir un recomendador explicable que filtre y ordene las aulas según disponibilidad, capacidad, recursos y tipo de actividad.
5. Verificar el sistema mediante pruebas automatizadas, pruebas de permisos y límites y casos de aceptación.

## Justificación

Un sistema centralizado reduce el tiempo que el profesor invierte en encontrar un espacio y elimina los conflictos de horario, porque la validación deja de depender de la memoria de una persona y pasa a ejecutarse en el servidor cada vez que se confirma una reserva. Para la institución, el historial de novedades del inventario y los reportes de uso convierten observaciones aisladas en información útil para planear mantenimiento y compras.

Se eligió una aplicación móvil porque profesores y encargados trabajan desplazándose entre bloques y salones; poder consultar, reservar o reportar un daño desde el teléfono, en el momento en que ocurre, es la condición para que los datos se mantengan actualizados.

## Alcance y limitaciones

La versión 1.2.1 cubre los doce requerimientos funcionales consolidados (Apéndice C) para una sede, con tres roles y una aplicación para Android. Incluye gestión de usuarios, aulas, inventario y novedades, horarios y bloqueos, búsqueda y recomendación de aulas, reservas con lista y calendario semanal, alertas y avisos sobre las reservas, y reportes. La API funciona en la nube, por lo que la aplicación se puede usar desde cualquier red con acceso a Internet.

Quedan fuera de esta versión la integración con el sistema académico o con un proveedor de identidad institucional, el trabajo sin conexión, la versión para iOS, los códigos QR y la predicción de demanda. Las alertas se consultan cada 30 segundos mientras la aplicación está abierta; no son notificaciones push, así que no llegan con la aplicación cerrada. La aplicación se distribuye como APK, fuera de la tienda de Google, y la API usa planes gratuitos: tras 15 minutos sin uso el servidor se suspende y la primera consulta posterior tarda cerca de un minuto.

## Marco de referencia

### Desarrollo móvil multiplataforma

Flutter es un kit de desarrollo de Google que compila una única base de código en Dart a aplicaciones nativas y ofrece componentes basados en Material Design (Google, s.f.). Su uso permitió construir la interfaz una sola vez y dejar abierta la posibilidad de generar una versión para iOS sin reescribir la lógica de presentación.

### Arquitectura REST

REST es un estilo arquitectónico en el que el cliente y el servidor se comunican mediante recursos identificados por URL, operaciones uniformes y mensajes sin estado (Fielding, 2000). En AulaGestión cada entidad del dominio se expone como recurso JSON y el contrato se documenta con la especificación OpenAPI 3.1 (OpenAPI Initiative, 2021), lo que permite probar la API de forma independiente a la aplicación.

### Autenticación basada en tokens

Laravel es un marco de trabajo PHP que provee enrutamiento, validación, ORM y migraciones (Laravel, s.f.-a). Su paquete Sanctum emite tokens de acceso personales para clientes móviles que el servidor valida en cada solicitud y puede revocar en cualquier momento (Laravel, s.f.-b). Este mecanismo evita mantener sesiones en el servidor y se adapta al modelo sin estado de REST.

### Persistencia relacional

MySQL 8 es un sistema gestor de bases de datos relacional con soporte para transacciones ACID, claves foráneas, índices compuestos y bloqueo de filas (Oracle, s.f.). Estas características sostienen las reglas más sensibles del proyecto: la integridad entre aulas, recursos y reservas, y la prevención de reservas concurrentes sobre el mismo espacio.

### Calidad, seguridad y protección de datos

Los requerimientos no funcionales se organizaron según las características del modelo de calidad de producto ISO/IEC 25010 (International Organization for Standardization [ISO], 2011). Las decisiones de seguridad del cliente móvil se contrastaron con los riesgos del OWASP Mobile Top 10, en especial el almacenamiento inseguro de credenciales y la comunicación sin cifrar (OWASP Foundation, 2024). Como el sistema guarda nombres y correos de docentes, su operación real debe ajustarse a la Ley 1581 de 2012 sobre protección de datos personales (Congreso de la República de Colombia, 2012).

## Metodología

El proyecto se gestionó con Scrum, un marco de trabajo que organiza el desarrollo en iteraciones cortas con roles, eventos y artefactos definidos para inspeccionar y adaptar el producto de forma continua (Schwaber y Sutherland, 2020). La Tabla 1 resume la distribución de roles y responsabilidades del equipo.

**Tabla 1**

*Roles del equipo Scrum*

| Integrante | Rol Scrum | Responsabilidad principal |
|---|---|---|
| María Alejandra Vallejo Zamora | Product Owner | Priorización del backlog y validación de criterios de aceptación |
| Juan David Delgado Muñoz | Scrum Master | Facilitación del proceso, integración y control de versiones |
| Julián Andrés Ceballos Eraso | Equipo de desarrollo | Backend, modelo de datos y reglas de negocio |
| David Esteban Ortiz Ortiz | Equipo de desarrollo | Aplicación Flutter, interfaces y experiencia de usuario |

El backlog del producto se construyó a partir de los doce requerimientos funcionales, cada uno con criterios mínimos de aceptación. Cada incremento debía cumplir una definición de terminado común: código integrado en la rama `develop`, validaciones del servidor, pruebas existentes en verde y documentación actualizada.

El control de versiones se llevó en Git con tres tipos de rama: `main` para la versión estable y demostrable, `develop` para la integración del sprint y `feature/*` para cada historia. Los cambios entraron mediante pull requests y mensajes de commit convencionales. Un flujo de integración continua en GitHub Actions (GitHub, s.f.) se ejecuta en cada push o pull request hacia `main` o `develop`: en el backend aplica el formateador Pint y la suite de pruebas, y en la aplicación ejecuta `dart format`, `flutter analyze` y `flutter test`.

## Requerimientos

### Requerimientos funcionales

La Tabla 2 presenta los doce requerimientos funcionales consolidados. Su relación con los componentes que los implementan y con la evidencia de verificación se encuentra en el Apéndice C.

**Tabla 2**

*Requerimientos funcionales*

| Código | Nombre | Descripción |
|---|---|---|
| RF01 | Autenticación y acceso | Inicio y cierre de sesión con correo y contraseña; solo las cuentas activas acceden y cada rol ve sus módulos. |
| RF02 | Usuarios y roles | Crear, actualizar y deshabilitar usuarios con rol de administrador, profesor o encargado de inventario. |
| RF03 | Gestión de aulas | Registrar código, nombre, ubicación, capacidad, tipo y estado; deshabilitación lógica sin perder historial. |
| RF04 | Inventario | Registrar recursos por aula con categoría, cantidad total, cantidad disponible y estado. |
| RF05 | Novedades de inventario | Registrar daños, faltantes, traslados, mantenimientos, recuperaciones y ajustes con usuario y fecha. |
| RF06 | Horarios y bloqueos | Definir jornadas por día y aula, y bloquear franjas por evento o mantenimiento. |
| RF07 | Disponibilidad | Determinar las aulas libres excluyendo cruces, bloqueos, aulas inactivas y franjas fuera de horario. |
| RF08 | Recomendación | Ordenar las aulas válidas con un porcentaje de coincidencia y las razones del resultado. |
| RF09 | Detalle y alternativas | Mostrar el detalle del aula, los recursos faltantes y franjas alternativas cuando no hay coincidencia exacta. |
| RF10 | Reservas | Confirmar y cancelar reservas con revalidación de disponibilidad y motivo obligatorio de cancelación. |
| RF11 | Calendario | Consultar las reservas en lista o vista semanal con filtros por estado y rango de fechas. |
| RF12 | Avisos y reportes | Alertas cuando el aula de una reserva deja de estar disponible, recordatorios, historial de avisos y reportes de uso de aulas y estado del inventario. |

## Requerimientos no funcionales

Los requerimientos no funcionales se agruparon por característica de calidad (ISO, 2011), como muestra la Tabla 3. Cada uno indica el mecanismo con el que se satisface en la versión entregada.

**Tabla 3**

*Requerimientos no funcionales*

| Código | Característica | Requerimiento y mecanismo |
|---|---|---|
| RNF01 | Seguridad | Contraseñas con hash; tokens Sanctum con vigencia de 12 horas, revocados al cerrar sesión o deshabilitar la cuenta; token guardado en almacenamiento seguro del dispositivo; el APK de versión solo se comunica por HTTPS. |
| RNF02 | Seguridad | Autorización por rol en el servidor mediante middleware, independiente de lo que muestre la interfaz; límite de 10 intentos de inicio de sesión y 60 solicitudes por minuto. |
| RNF03 | Fiabilidad | Confirmación de reservas en una transacción con bloqueo de fila del aula; ninguna reserva superpuesta puede persistir. |
| RNF04 | Fiabilidad | Validación de todos los datos críticos en el servidor con respuestas JSON uniformes (HTTP 422 para datos inválidos, 403 sin permiso, 409 en conflicto). |
| RNF05 | Usabilidad | Interfaz en español con Material 3; formularios con ejemplos, ayudas y listas cerradas; indicador de carga en cada acción que espera al servidor; textos legibles hasta 1,6 veces el tamaño de letra del sistema; mensajes claros ante errores de red, sin cierres inesperados. |
| RNF06 | Mantenibilidad | Separación en capas (pantallas, repositorios, cliente HTTP; controladores y servicios), formato y análisis estático automáticos en la integración continua. |
| RNF07 | Trazabilidad | Registro de auditoría con usuario, acción, valores anteriores y nuevos, IP y fecha para aulas, inventario, horarios y reservas. |
| RNF08 | Portabilidad | Aplicación Android generada con Flutter; API empaquetada en una imagen Docker; base de datos reproducible con migraciones, datos iniciales y Docker Compose. |
| RNF09 | Disponibilidad | API publicada en Render y base de datos MySQL en Aiven con conexión SSL; la aplicación espera hasta 75 segundos para que el servidor gratuito despierte. |

## Roles y permisos

El sistema define tres roles. La Tabla 4 resume las operaciones permitidas a cada uno; el servidor aplica estas reglas en cada endpoint, de modo que una llamada directa a la API sin el rol adecuado recibe HTTP 403.

**Tabla 4**

*Permisos por rol*

| Módulo u operación | Administrador | Profesor | Inventario |
|---|---|---|---|
| Consultar aulas, inventario y notificaciones | Sí | Sí | Sí |
| Buscar, recomendar y reservar aulas | Sí | Sí | No |
| Cancelar reservas | Todas | Propias | No |
| Registrar recursos y novedades de inventario | Sí | No | Sí |
| Consultar reportes | Sí | No | Sí |
| Gestionar usuarios, aulas, horarios y bloqueos | Sí | No | No |

## Reglas de negocio

Las siguientes reglas se implementan en el servidor y se verifican en las pruebas:

1. Solo las cuentas activas pueden iniciar sesión.
2. Un aula inactiva, en mantenimiento o fuera de servicio no se puede reservar.
3. La capacidad del aula debe ser igual o mayor que el número de asistentes.
4. La franja solicitada debe estar dentro de una jornada configurada para ese día y no coincidir con un bloqueo.
5. Dos reservas se superponen cuando el inicio existente es anterior al final solicitado y el final existente es posterior al inicio solicitado.
6. Una reserva dura entre 30 minutos y 8 horas.
7. Los recursos obligatorios deben existir en el aula, estar disponibles y tener cantidad disponible mayor que cero.
8. La disponibilidad se revalida al confirmar, dentro de una transacción con bloqueo de fila.
9. Solo el dueño de la reserva o el administrador puede cancelarla, nunca después de iniciada y siempre con un motivo de al menos cinco caracteres; la franja queda libre de inmediato.

10. No se puede crear un bloqueo que afecte una reserva confirmada.
11. La cantidad disponible de un recurso se mantiene entre cero y su cantidad total.
12. Todo cambio crítico de aulas, inventario, horarios o reservas genera un registro de auditoría.
13. El código de un aula sigue el formato letras o números, guion y número (por ejemplo, A-101), y su tipo se elige de un catálogo cerrado.
14. Toda aula nueva recibe una jornada inicial de lunes a sábado, de 7:00 a 19:00, para que aparezca en las búsquedas desde el primer momento.
15. Si el aula de una reserva vigente queda deshabilitada, en mantenimiento, fuera de servicio o bloqueada, el profesor recibe una alerta; cuando el aula se recupera, la alerta se resuelve y llega un aviso.
16. Cuando falta menos de un día para una reserva, el profesor recibe un recordatorio una sola vez, salvo que la haya creado en la última hora.

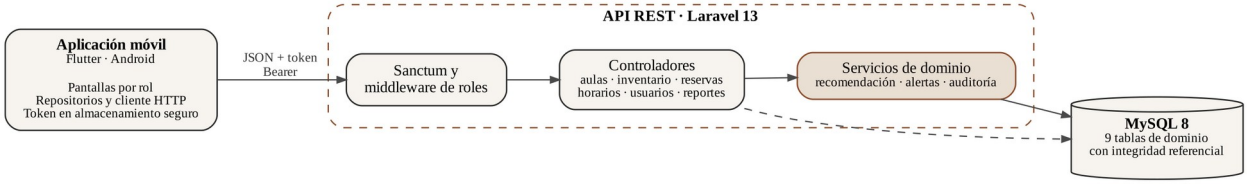
## Diseño de la solución

### Arquitectura

La solución se compone de tres capas desacopladas (Figura 1). La aplicación Flutter presenta la interfaz, valida formularios, guarda el token de forma segura y consume la API. La API Laravel verifica identidad, permisos, integridad y concurrencia, y delega la lógica de dominio en servicios. MySQL conserva la información y garantiza la integridad referencial. Como la lógica crítica no depende de la interfaz, el servidor vuelve a validar cada regla aunque alguien llame a la API directamente.

**Figura 1**

*Arquitectura cliente-servidor de AulaGestión*

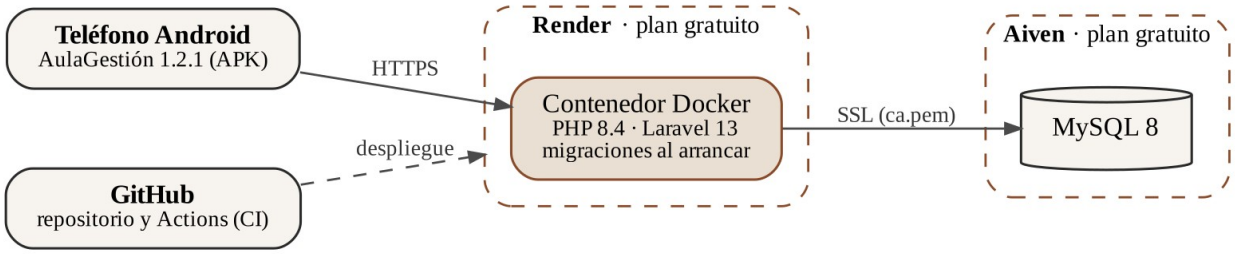


## Despliegue

La API se publica en Render como un contenedor Docker con PHP 8.4, que aplica las migraciones pendientes cada vez que arranca. La base de datos es un servicio MySQL 8 de Aiven, al que la API se conecta por SSL con el certificado de la entidad emisora. Ambos usan planes gratuitos. Las variables de configuración, como la llave de la aplicación y las credenciales de la base de datos, se definen en el panel de Render y nunca se guardan en el repositorio. La Figura 2 muestra cómo se conectan los componentes en producción.

**Figura 2**

*Diagrama de despliegue*

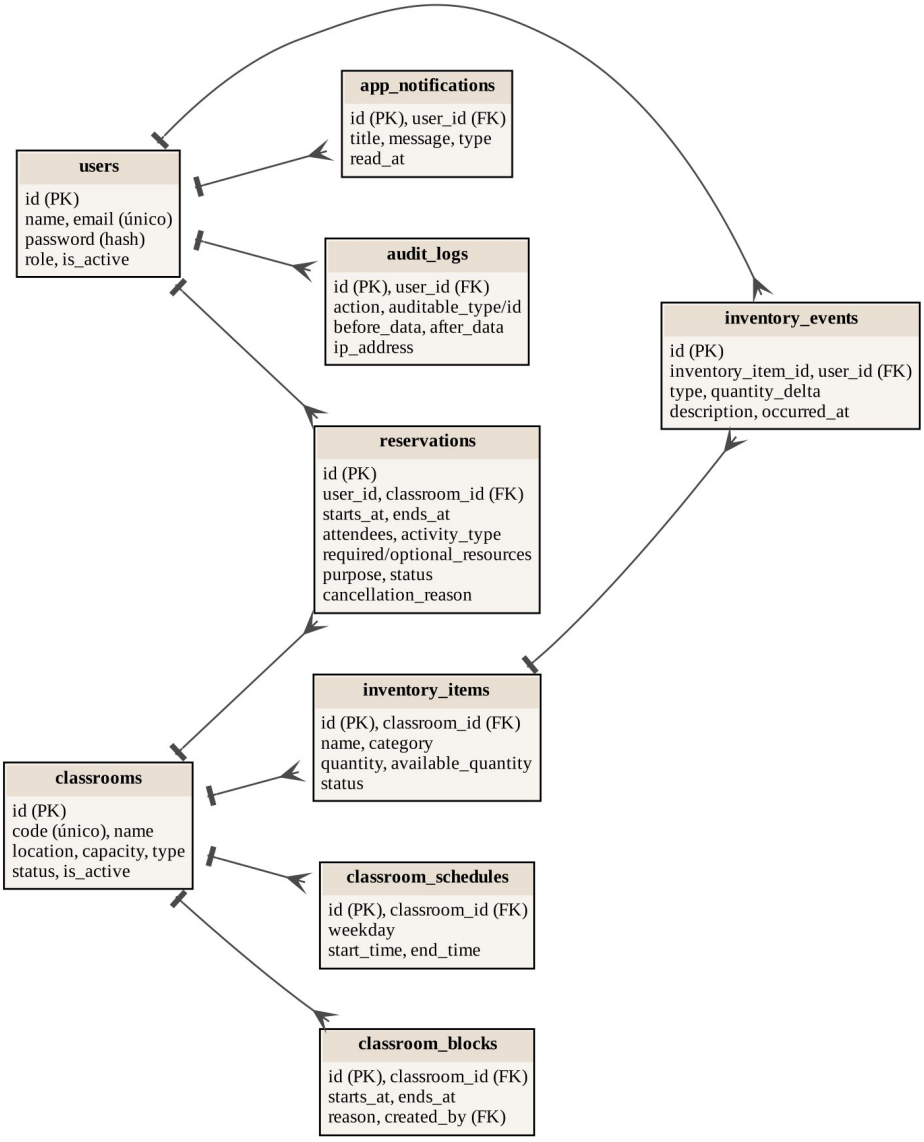



## Modelo de datos

El modelo relacional (Figura 3) gira alrededor de dos entidades: `users` y `classrooms`. Un aula contiene recursos de inventario, jornadas habilitadas y bloqueos, y recibe reservas; cada recurso acumula un historial de novedades. Las reservas no se eliminan: cambian de estado, lo que conserva la trazabilidad. Un índice compuesto sobre aula, inicio, fin y estado acelera la consulta de superposición.

**Figura 3**

*Modelo entidad-relación*




*Nota.* Se muestran los atributos principales; todas las tablas incluyen marcas de tiempo de creación y actualización.

Los estados de cada entidad se controlan con dominios cerrados, que se resumen en la Tabla 5.

**Tabla 5**

*Dominios de estado del modelo*

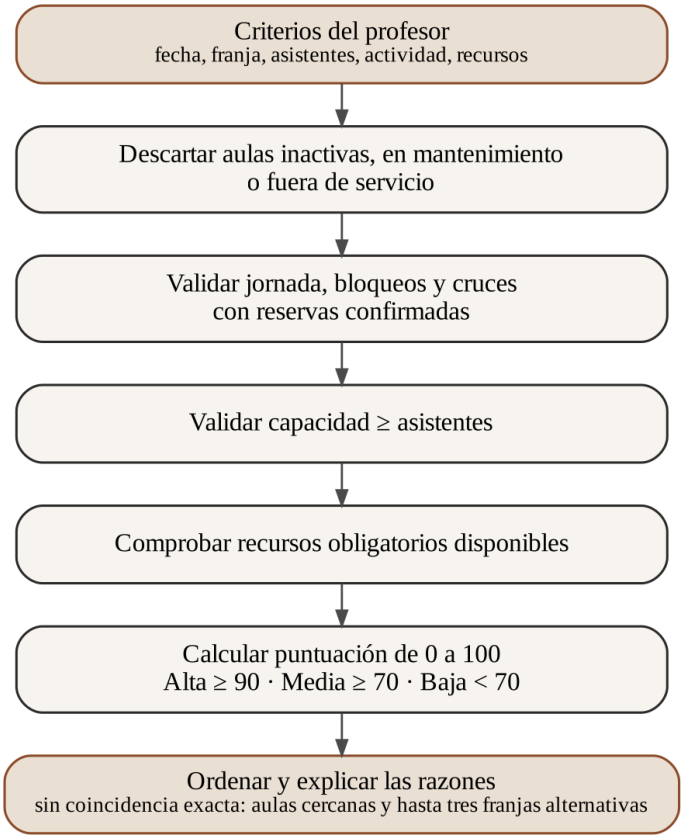
| Entidad | Valores permitidos |
|---|---|
| Usuario (rol) | admin, professor, inventory |
| Aula | available, maintenance, out_of_service |
| Recurso de inventario | available, loaned, maintenance, damaged, out_of_service |
| Novedad de inventario | damage, missing, transfer, maintenance, recovery, adjustment |
| Reserva | confirmed, cancelled, completed, rejected |
| Notificación (tipo) | info, classroom_unavailable, classroom_restored, reservation_reminder |
| Tipo de aula | Clase interactiva, Multimedia, Informática, Laboratorio, Actividad lúdica, Auditorio |

## Recomendador de aulas

El recomendador está implementado en `RecommendationService` y funciona en dos etapas (Figura 4). Primero descarta las aulas que no pueden atender la solicitud; después asigna a las restantes una puntuación explicable. No se trata de un modelo de inteligencia artificial sino de reglas transparentes y medibles, adecuadas para el alcance del proyecto y fáciles de auditar.

**Figura 4**

*Flujo del recomendador de aulas*




La puntuación parte de una base y suma componentes hasta un máximo de 100 puntos, según la Tabla 6. El resultado se clasifica como coincidencia alta (90 o más), media (70 a 89) o baja (menos de 70), y la aplicación muestra el porcentaje junto con las razones que lo explican.

**Tabla 6**

*Componentes de la puntuación de recomendación*

| Componente | Puntos | Cálculo |
|---|---|---|
| Base por aula válida | 55 | Se asigna a toda aula que superó los filtros. |
| Ajuste de capacidad | 0 a 25 | 25 menos la mitad de la diferencia entre capacidad y asistentes; premia el aula de tamaño más cercano. |
| Recursos obligatorios | 0 a 10 | Proporción de recursos obligatorios presentes. |
| Recursos opcionales | 0 a 5 | Proporción de recursos opcionales presentes. |
| Afinidad de tipo | 0 o 5 | El tipo de aula es adecuado para la actividad, según la Tabla 7. |

| Componente | Puntos | Cálculo |
|---|---|---|
| Penalización | −20 c/u | Por cada recurso obligatorio faltante; el total se acota entre 0 y 100. |

El tipo de actividad y el tipo de aula se eligen de listas cerradas, de modo que la afinidad se calcula con una tabla fija y no con texto libre. La Tabla 7 muestra qué tipos de aula se consideran adecuados para cada actividad.

**Tabla 7**

*Tipos de aula adecuados para cada actividad*

| Actividad | Tipos de aula adecuados |
|---|---|
| Clase | Clase interactiva, Multimedia |
| Taller | Clase interactiva, Actividad lúdica |
| Presentación o video | Multimedia, Auditorio |
| Trabajo en computadores | Informática |
| Práctica de laboratorio | Laboratorio |
| Actividad lúdica | Actividad lúdica |
| Reunión o evento | Auditorio, Clase interactiva |

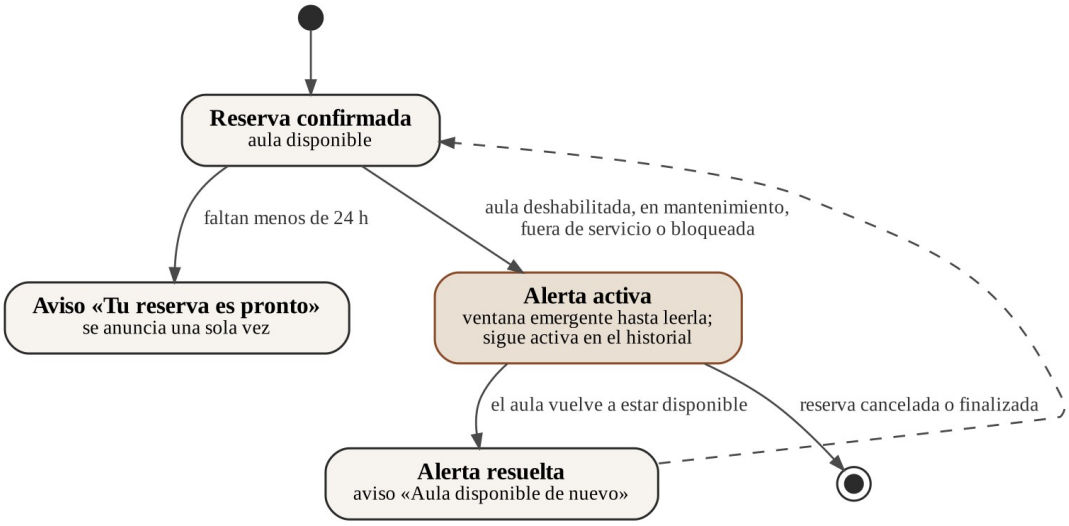
### Alertas de disponibilidad

Una reserva confirmada puede quedar sin efecto si, después de hacerla, el aula se deshabilita, pasa a mantenimiento o queda bloqueada. Para que el profesor no se entere al llegar al salón, el servicio `ReservationAlertService` compara cada reserva vigente con el estado real del aula. La comparación se hace al consultar y no al editar el aula; así se detectan también los cambios hechos directamente en la base de datos.

Mientras la aplicación está abierta, consulta las alertas cada 30 segundos; en segundo plano deja de hacerlo para no gastar datos. Una alerta activa aparece como ventana emergente hasta que el profesor la lee, y sigue marcada como activa en el historial. Si el aula vuelve a estar disponible, la alerta se resuelve sola y llega un aviso breve que no interrumpe; si la reserva se cancela o termina, la alerta se cierra sin aviso. La Figura 5 resume este ciclo.

**Figura 5**

*Ciclo de las alertas y avisos de una reserva*




## Interfaz de usuario

La aplicación usa Material 3 con una paleta institucional configurable y navegación por pestañas. El menú de módulos se construye según el rol de la sesión, de modo que cada usuario ve solo lo que puede usar. La Tabla 8 describe las pantallas principales.

**Tabla 8**

*Pantallas principales de la aplicación*

| Pantalla | Roles | Función |
|---|---|---|
| Inicio de sesión | Todos | Autenticación con correo y contraseña. |
| Inicio | Todos | Para el profesor, su próxima reserva y las alertas activas; para el administrador, el resumen numérico del sistema. |
| Módulos | Todos | Menú de funciones filtrado por permisos. |
| Buscar y reservar | Profesor, administrador | Criterios de búsqueda, resultados con puntuación, detalle del aula, alternativas y confirmación. |
| Reservas | Profesor, administrador | Lista y calendario semanal con filtros; cancelación con motivo. |
| Inventario | Todos (gestión: inventario, administrador) | Recursos por aula, registro de novedades e historial. |
| Aulas | Todos (gestión: administrador) | Listado, detalle, creación con código y tipo guiados, edición, deshabilitación y rehabilitación. |

| Pantalla | Roles | Función |
|---|---|---|
| Usuarios, horarios y bloqueos | Administrador | Administración de cuentas, jornadas y franjas bloqueadas. |
| Reportes | Administrador, inventario | Uso de aulas y estado del inventario. |
| Notificaciones | Todos | Historial de alertas y avisos con su estado (activa, resuelta, nuevo, leído) y contador en la barra inferior. |
| Perfil | Todos | Datos de la cuenta, versión de la aplicación y cierre de sesión. |

### Interfaz de programación

La API expone sus recursos bajo el prefijo `/api` y responde en JSON. La Tabla 9 resume los endpoints; la especificación completa está en `docs/openapi.yaml` y puede ejecutarse con la colección de Postman incluida en el repositorio.

**Tabla 9**

*Endpoints de la API REST*

| Método | Ruta | Acceso |
|---|---|---|
| GET | /health | Público |
| POST | /auth/login | Público (10 solicitudes por minuto) |
| GET, POST | /auth/me, /auth/logout | Autenticado |
| GET | /dashboard | Autenticado |
| GET | /classrooms, /classrooms/{id} | Autenticado |
| POST, PUT, DELETE | /classrooms, /classrooms/{id} | Administrador |
| GET | /inventory, /inventory/{id} | Autenticado |
| POST, PUT | /inventory, /inventory/{id} | Inventario, administrador |
| GET, POST | /inventory-events, /inventory/{id}/events | Inventario, administrador |
| POST | /recommendations/search | Profesor, administrador |
| GET | /reservations, /reservations/{id} | Autenticado (según propiedad) |
| POST, PATCH | /reservations, /reservations/{id}/cancel | Profesor, administrador |
| GET, POST, PUT, DELETE | /schedules, /blocks | Administrador |
| CRUD | /users | Administrador |
| GET, PATCH | /notifications, /notifications/{id}/read, /notifications/read-all | Autenticado |
| GET | /notifications/alerts | Autenticado |

| Método | Ruta | Acceso |
|---|---|---|
| GET | /reports/summary | Inventario, administrador |

## Implementación

### Tecnologías

La Tabla 10 lista las tecnologías empleadas y la función de cada una en el proyecto.

**Tabla 10**

*Tecnologías utilizadas*

| Tecnología | Versión | Uso |
|---|---|---|
| Flutter / Dart | 3.35+ / 3.6+ | Aplicación móvil Android |
| http, flutter_secure_storage, intl | 1.5, 9.2, 0.20 | Cliente HTTP, almacenamiento seguro del token y formato de fechas |
| Laravel / PHP | 13 / 8.3+ (8.4 en producción) | API REST, validación, ORM y migraciones |
| Laravel Sanctum | — | Emisión y revocación de tokens |
| MySQL | 8 | Base de datos relacional |
| Docker y Docker Compose | — | Imagen de producción de la API; MySQL y phpMyAdmin en desarrollo |
| Render | Plan gratuito | Hospedaje de la API con HTTPS |
| Aiven | Plan gratuito | MySQL 8 administrado con conexión SSL |
| PHPUnit, flutter_test | — | Pruebas automatizadas |
| Pint, flutter_lints | — | Formato y análisis estático |
| GitHub Actions | — | Integración continua |
| OpenAPI 3.1, Postman | — | Documentación y prueba de la API |

### Organización del código

El repositorio separa el backend, la aplicación y la documentación:

```
aula_gestion/
├── backend/      API Laravel (app, database, routes, tests)
├── mobile/lib/    core, models, repositories, screens, widgets
├── docs/         OpenAPI, guía técnica, pruebas y este documento
├── postman/      colección y entorno de la API
├── .github/      flujo de integración continua
└── docker-compose.yml
```

En la aplicación, las pantallas no llaman directamente a la red: lo hacen a través de repositorios (`AuthRepository`, `SchoolRepository`) que usan un cliente HTTP común, encargado de adjuntar el token y traducir los errores del servidor a mensajes comprensibles. Cada vez que se guarda un cambio, el cliente avisa a las pantallas visibles para que se actualicen sin recargar. Las listas de opciones de los formularios están en `catalogs.dart` y deben coincidir con las del servidor. En el servidor, los controladores validan la solicitud y delegan en los modelos y en los servicios `RecommendationService`, `ReservationAlertService` y `AuditService`.

**Seguridad**

Las contraseñas se almacenan con hash y los tokens tienen una vigencia de 12 horas; se revocan al cerrar sesión y al deshabilitar la cuenta. En el dispositivo el token se guarda con `flutter_secure_storage`, que usa el almacén cifrado del sistema operativo, siguiendo la recomendación de no dejar credenciales en almacenamiento plano (OWASP Foundation, 2024). El servidor limita la frecuencia de solicitudes, comprueba el rol en cada ruta protegida, registra la IP en la auditoría y no guarda secretos en el repositorio.

El APK de versión solo acepta conexiones HTTPS; el tráfico HTTP queda permitido únicamente en la compilación de desarrollo, para probar contra un servidor local. La conexión entre la API y MySQL viaja cifrada con SSL. El archivo `.env` no se sube al repositorio: se publica una plantilla comentada y, en producción, los valores viven en el panel de Render. Antes de un uso real con la institución deben cambiarse las contraseñas de los usuarios de demostración.

## Pruebas

La verificación combinó pruebas automatizadas, que se ejecutan en cada integración, con casos de aceptación manuales sobre la aplicación instalada y pruebas de permisos y límites sobre la API.

### Pruebas automatizadas

La Tabla 11 describe las pruebas automatizadas incluidas en el repositorio. Se ejecutan con `php artisan test` en el backend y con `flutter analyze` y `flutter test` en la aplicación.

**Tabla 11**

*Pruebas automatizadas*

| Archivo | Verificación |
|---|---|
| backend/tests/Feature/AuthTest.php | Un usuario activo recibe token y rol; un usuario deshabilitado no puede iniciar sesión. |
| backend/tests/Feature/ RecommendationTest.php | Un aula sin un recurso obligatorio no aparece como coincidencia exacta. |
| backend/tests/Feature/ ReservationConflictTest.php | Una reserva superpuesta devuelve HTTP 409 y no crea un segundo registro. |
| backend/tests/Feature/ ClassroomRegistrationTest.php (5 casos) | Un aula nueva aparece en las búsquedas sin pasos extra; el código respeta el formato; el tipo sale del catálogo; la afinidad de la actividad suma puntos. |
| backend/tests/Feature/ ReservationAlertTest.php (12 casos) | Ciclo completo de alertas: creación sin duplicados, cambios hechos directo en la base de datos, resolución, aviso de aula recuperada, recordatorio único, bloqueos que se cruzan y aislamiento entre usuarios. |
| mobile/test/domain_models_test.dart | Conversión segura del JSON de aulas, inventario y recomendaciones. |
| mobile/test/ui_behavior_test.dart | Pantallas sin desbordes en varios anchos y tamaños de letra; botones con indicador de carga y sin doble envío; errores mostrados dentro del formulario; inicio según el rol; alertas emergentes y avisos; pausa de las consultas en segundo plano. |

### Casos de aceptación

La Tabla 12 recoge los doce casos de aceptación ejecutados sobre la aplicación con los datos iniciales del sistema.

**Tabla 12**

*Casos de aceptación*

| Caso | Escenario | Resultado esperado |
|---|---|---|
| CA-01 | Iniciar sesión con los tres usuarios de demostración | Cada rol ve únicamente sus módulos |

| Caso | Escenario | Resultado esperado |
|---|---|---|
| CA-02 | Como administrador, crear el aula D-201 con capacidad 32 | El aula aparece con ubicación, tipo y estado |
| CA-03 | Como inventario, agregar un videobeam a D-201 y actualizar su cantidad | El recurso queda actualizado |
| CA-04 | Buscar una franja en la que un aula está ocupada o bloqueada | El aula no aparece como coincidencia |
| CA-05 | Buscar para 25 estudiantes con videobeam obligatorio | Resultados ordenados con porcentaje y razones |
| CA-06 | Reservar de nuevo la misma aula y franja | Conflicto HTTP 409, sin registro duplicado |
| CA-07 | Cancelar una reserva futura con motivo y repetir la búsqueda | Reserva cancelada y franja disponible |
| CA-08 | Registrar un daño y revisar el historial | Se conservan usuario, fecha y descripción |
| CA-09 | Detener el servidor y actualizar una lista en la aplicación | Mensaje de red sin cierre inesperado |
| CA-10 | Instalar el APK 1.2.1 en un teléfono y entrar con la API pública | La aplicación funciona con datos móviles o cualquier red Wi-Fi |
| CA-11 | Poner en mantenimiento el aula de una reserva vigente y luego habilitarla | Aparece la alerta emergente; al habilitarla se resuelve y llega el aviso de aula disponible |
| CA-12 | Abrir la aplicación con una reserva que empieza en menos de 24 horas | Se muestra una vez el aviso «Tu reserva es pronto» |

En la ejecución final de la versión 1.2.1, las 21 pruebas del servidor y las pruebas de la aplicación resultaron aprobadas.

### Pruebas de permisos y límites

Con Postman se comprobó que un profesor recibe HTTP 403 al intentar crear usuarios, que el encargado de inventario no puede crear horarios, que un profesor no accede a reservas ajenas y que una cuenta deshabilitada pierde sus tokens. En cuanto a límites, el servidor responde HTTP 422 ante cero asistentes, capacidades superiores a 1000, hora final igual o anterior a la inicial, reservas de menos de 30 minutos o más de 8 horas, cantidades disponibles mayores que la total y motivos de cancelación de menos de cinco caracteres.

## Resultados

El proyecto entrega la versión 1.2.1 de AulaGestión con los doce requerimientos funcionales implementados y verificables. La Tabla 13 relaciona los productos de la entrega y su ubicación en el repositorio.

**Tabla 13**

*Productos entregados*

| Producto | Ubicación |
|---|---|
| Instalador Android 1.2.1 | mobile/build/app/outputs/flutter-apk/AulaGestion-1.2.1.apk |
| API pública | https://aulagestion-api.onrender.com/api |
| Código fuente de la aplicación | mobile/lib/ |
| Código fuente de la API y migraciones | backend/ |
| Especificación de la API | docs/openapi.yaml |
| Colección y entorno de Postman | postman/ |
| Guía técnica y plan de pruebas | docs/ |
| Entorno de base de datos local | docker-compose.yml |
| Imagen de producción de la API | backend/Dockerfile |
| Historial de versiones | CHANGELOG.md |

Frente a la situación inicial, el sistema impide por diseño las reservas superpuestas, hace visible el estado de cada recurso con su historial y reduce la búsqueda de un aula a indicar fecha, franja, asistentes y recursos. La recomendación explica sus resultados, lo que facilita que el profesor confíe en la sugerencia o elija otra opción con información suficiente. Además, si algo cambia en el aula después de reservar, el profesor se entera desde la aplicación y no al llegar al salón. El Apéndice D muestra la aplicación en funcionamiento con los tres roles.

El producto creció en cinco versiones, resumidas en la Tabla 14.

**Tabla 14**

*Historial de versiones*

| Versión | Fecha | Cambios principales |
|---|---|---|
| 1.0.0 | 7 sep. 2026 | Primera versión con los doce requerimientos funcionales, pruebas, OpenAPI y Postman. |
| 1.0.1 | 27 oct. 2026 | APK instalable, correcciones de la barra superior y formato del código para la integración continua. |
| 1.1.0 | 2 nov. 2026 | API publicada en Render y MySQL en Aiven; el APK de versión pasa a usar solo HTTPS. |
| 1.2.0 | 8 nov. 2026 | Alertas de aulas no disponibles, recordatorios, inicio según el rol, formularios guiados con catálogos y rehabilitación de aulas. |

| Versión | Fecha | Cambios principales |
|---|---|---|
| 1.2.1 | 8 nov. 2026 | Aviso breve cuando un aula se recupera y ajustes al conteo de no leídas y al trabajo en segundo plano. |

## Conclusiones

La separación entre aplicación, API y base de datos permitió desarrollar en paralelo la interfaz y las reglas de negocio, y garantizó que las validaciones críticas se ejecuten en el servidor sin depender del cliente. Esta decisión fue la base para que permisos, capacidad, horarios y cruces se cumplan aun ante llamadas directas a la API.

Resolver la concurrencia con una transacción y un bloqueo de fila sobre el aula, junto con la revalidación al confirmar, fue más simple y confiable que detectar conflictos después de guardar. Las pruebas automatizadas de superposición convirtieron esta regla en una garantía verificable en cada integración.

Un recomendador basado en reglas fue suficiente para el problema planteado. Al mostrar el porcentaje y sus razones, la recomendación se vuelve auditable y comprensible para el usuario, y sus pesos pueden ajustarse sin cambiar la arquitectura.

Calcular las alertas al consultar, y no al momento de editar un aula, resultó más robusto: el sistema reacciona igual sin importar por dónde llegó el cambio. Publicar la API en servicios gratuitos permitió probar la aplicación en teléfonos reales fuera de la red local, a cambio de una primera respuesta lenta cuando el servidor está dormido.

Scrum, el flujo de ramas con pull requests y la integración continua mantuvieron estable la rama principal y dieron al equipo una definición de terminado compartida. Como trabajo futuro se propone pasar a un plan de hospedaje sin suspensión, convertir las alertas en notificaciones push, exportar reportes a PDF y Excel, incorporar códigos QR de ingreso a las aulas, soportar varias sedes e integrar el calendario institucional; la arquitectura actual permite agregar estos módulos sin reescribir el núcleo de reservas.

## Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.ª ed.). https://doi.org/10.1037/0000165-000

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial n.º 48.587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

GitHub. (s.f.). *GitHub Actions documentation*. https://docs.github.com/en/actions

Google. (s.f.). *Flutter documentation*. https://docs.flutter.dev

International Organization for Standardization. (2011). *Systems and software engineering: Systems and software Quality Requirements and Evaluation (SQuaRE): System and software quality models* (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

Laravel. (s.f.-a). *Laravel 13.x documentation*. https://laravel.com/docs/13.x

Laravel. (s.f.-b). *Laravel Sanctum*. https://laravel.com/docs/13.x/sanctum

OpenAPI Initiative. (2021). *OpenAPI Specification v3.1.0*. https://spec.openapis.org/oas/v3.1.0

Oracle. (s.f.). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

OWASP Foundation. (2024). *OWASP Mobile Top 10*. https://owasp.org/www-project-mobile-top-10/

Schwaber, K. y Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/scrum-guide.html

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

# Apéndice A

## Manual de instalación y despliegue

### Requisitos

Se requiere Git, Docker Desktop (o MySQL 8 instalado), PHP 8.3 o superior, Composer 2, Flutter estable con Android Studio y un emulador Android o un teléfono con depuración USB. Antes de compilar, `flutter doctor` no debe mostrar errores.

### Uso con la API pública

Para usar la aplicación no hace falta instalar nada más: basta con instalar el APK `AulaGestion-1.2.1.apk` en un teléfono Android (permitiendo la instalación desde fuentes externas) e iniciar sesión. La aplicación se conecta a `https://aulagestion-api.onrender.com/api`. Si el servidor estaba inactivo, la primera consulta tarda cerca de un minuto. Las secciones siguientes explican cómo ejecutar el sistema en un equipo propio.

### Base de datos local

Desde la carpeta raíz del proyecto:

```
docker compose up -d
docker compose ps
```

MySQL queda disponible en `127.0.0.1:3307` y phpMyAdmin en `http://localhost:8081`. El volumen `aula_mysql_data` conserva los datos aunque se detengan los contenedores.

### API Laravel local

```
cd backend
cp .env.example .env   # PowerShell: Copy-Item .env.example .env
composer install
php artisan key:generate
php artisan migrate --seed
php artisan serve --host=0.0.0.0 --port=8000
```

La dirección `http://127.0.0.1:8000/api/health` debe responder `success: true`. Para reiniciar los datos en desarrollo se usa `php artisan migrate:fresh --seed`, que elimina y reconstruye las tablas.

**Aplicación móvil**

```
cd mobile
flutter pub get
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8000/api
```

Esta es la compilación de desarrollo, la única que acepta HTTP.

En el emulador, `10.0.2.2` apunta al computador anfitrión. En un teléfono físico, ambos equipos deben estar en la misma red Wi-Fi y se reemplaza esa dirección por la IP local del computador (por ejemplo, `http://192.168.1.20:8000/api`), habilitando el puerto 8000 en el firewall si es necesario.

**Despliegue en Render y Aiven**

1. En Aiven, crear un servicio MySQL del plan gratuito y descargar su certificado `ca.pem`.
2. En Render, crear un Web Service desde el repositorio con directorio raíz `backend`, entorno Docker y plan gratuito.
3. En Environment, definir `APP_ENV=production`, `APP_DEBUG=false`, `APP_KEY`, `LOG_CHANNEL=stderr`, las variables `DB_*` con los datos de Aiven, `MYSQL_ATTR_SSL_CA=/etc/secrets/ca.pem` y, solo en el primer despliegue, `SEED_DEMO=true`.
4. En Secret Files, subir el archivo `ca.pem`.
5. Desplegar y comprobar que `/api/health` responda.

**Generación del APK**

```
flutter build apk --release \
  --dart-define=API_BASE_URL=https://aulagestion-api.onrender.com/api
```

El instalador queda en `mobile/build/app/outputs/flutter-apk/app-release.apk`. La URL de la API se fija al compilar y debe empezar por `https://`, porque el APK de versión no acepta HTTP.

**Usuarios de demostración**

Los datos iniciales crean las cuentas de la Tabla A1.

**Tabla A1**

*Usuarios creados por los datos iniciales*

| Rol | Correo | Contraseña |
|---|---|---|
| Administrador | admin@demo.edu.co | Demo2026* |
| Profesor | profesor@demo.edu.co | Demo2026* |
| Inventario | inventario@demo.edu.co | Demo2026* |

*Nota.* Las cuentas son ficticias. Los datos iniciales también crean cinco aulas, inventario tecnológico, horarios de lunes a sábado, una reserva futura y notificaciones. Las contraseñas deben cambiarse antes de cualquier uso distinto de la demostración.

**Solución de problemas**

La Tabla A2 reúne los problemas más comunes durante la instalación.

**Tabla A2**

*Problemas frecuentes y solución*

| Síntoma | Solución |
|---|---|
| La primera consulta tarda cerca de un minuto | Es normal en el plan gratuito de Render: el servidor se suspende tras 15 minutos sin uso y despierta con la primera petición. |
| La aplicación indica que no hay conexión con el servidor | Confirmar que Laravel está en ejecución y que `/api/health` responde; usar `10.0.2.2` en el emulador o la IP local en el teléfono; revisar el firewall y la red. |
| Laravel no conecta con MySQL | Ejecutar `docker compose ps`, confirmar el puerto 3307 y las credenciales en `.env`, y limpiar la configuración con `php artisan config:clear`. |
| No aparecen recomendaciones | Verificar que la fecha sea de lunes a sábado y que el aula tenga horario, capacidad, estado disponible y los recursos obligatorios con cantidad disponible. El recurso debe registrarse con el mismo nombre que ofrece el buscador (por ejemplo, Videobeam). |
| Flutter no compila Android | Ejecutar `flutter doctor --android-licenses`, luego `flutter clean` y `flutter pub get`. |

# Apéndice B

## Manual de usuario

### Profesor

Al iniciar sesión, la pantalla de inicio muestra la próxima reserva y cualquier alerta activa.

**Buscar y reservar un aula.**

1. Iniciar sesión y abrir **Módulos > Buscar y reservar**.
2. Elegir fecha, hora inicial y hora final.
3. Indicar el número de estudiantes y el tipo de actividad.
4. Marcar los recursos obligatorios y, si se desea, los opcionales.
5. Pulsar **Buscar mejores aulas** y revisar porcentaje, capacidad, ubicación y razones de cada resultado.
6. Pulsar **Reservar esta aula**, escribir el motivo (mínimo cinco caracteres) y confirmar.

Si ninguna aula coincide por completo, la aplicación muestra aulas cercanas con los recursos que les faltan y hasta tres franjas alternativas.

**Atender una alerta.** Si el aula reservada deja de estar disponible, aparece una ventana emergente con el motivo. La reserva queda marcada en la lista; lo recomendable es cancelarla y buscar otra aula. Si el aula se recupera, la alerta se cierra sola y llega un aviso. Todas las alertas y avisos quedan en **Notificaciones**.

**Consultar o cancelar una reserva.**

1. Abrir **Reservas** y alternar entre lista y calendario semanal.
2. Filtrar por confirmadas, canceladas o finalizadas.
3. En una reserva futura confirmada, pulsar **Cancelar reserva**, registrar el motivo y confirmar. La franja queda disponible de inmediato.

### Encargado de inventario

**Registrar un recurso.**

1. Abrir **Gestión de inventario** y pulsar **Nuevo elemento**.

2. Seleccionar el aula, elegir la categoría de la lista e ingresar nombre, cantidad total, cantidad disponible y estado. Para que el buscador tenga en cuenta el recurso, use uno de los nombres sugeridos: Videobeam, Televisor, Computador, Parlantes, Aire acondicionado, Tablero acrílico o Microscopio.
3. Guardar.

**Registrar una novedad.**

1. Ubicar el recurso y pulsar **Registrar novedad**.
2. Seleccionar el tipo (daño, faltante, traslado, mantenimiento, recuperación o ajuste) y el nuevo estado.
3. Indicar el cambio en unidades disponibles: negativo para descontar, positivo para recuperar.
4. Describir lo ocurrido y registrar. El evento queda en **Novedades e historial** con usuario y fecha.

**Administrador**

1. **Usuarios:** registrar nombre, correo, contraseña temporal, rol y estado; deshabilitar cuentas sin borrar su historial.
2. **Aulas:** registrar código con el formato A-101, nombre, ubicación, capacidad, tipo (de la lista) y estado; usar mantenimiento o fuera de servicio para impedir reservas. El aula queda con horario de lunes a sábado de 7:00 a 19:00. Las aulas deshabilitadas se pueden ver y volver a habilitar desde la misma pantalla. Los profesores con reservas afectadas reciben una alerta automática.
3. **Horarios:** crear al menos una jornada para cada día reservable de cada aula.
4. **Bloqueos:** registrar franjas no disponibles por mantenimiento o evento; el sistema rechaza bloqueos que afecten reservas confirmadas.
5. **Reportes:** consultar usuarios y aulas activos, reservas del periodo, aulas más usadas y estado del inventario.

# Apéndice C

## Matriz de trazabilidad

La Tabla C1 relaciona cada requerimiento funcional con los componentes que lo implementan y la evidencia que lo verifica.

**Tabla C1**

*Trazabilidad de requerimientos funcionales*

| Requerimiento | Backend | Aplicación | Evidencia |
|---|---|---|---|
| RF01 Autenticación | AuthController, Sanctum, EnsureRole | Inicio de sesión, sesión segura, menú por rol | AuthTest; CA-01 |
| RF02 Usuarios | UserController | Pantalla de usuarios | Crear, actualizar y deshabilitar; prueba 403 |
| RF03 Aulas | ClassroomController | Pantalla de aulas | ClassroomRegistrationTest; CA-02 |
| RF04 Inventario | InventoryController | Pantalla de inventario | CA-03 |
| RF05 Novedades | InventoryEvent, AuditService | Formulario e historial | CA-08 |
| RF06 Horarios y bloqueos | ScheduleController | Horarios y bloqueos | CA-04; bloqueo sobre reserva rechazado |
| RF07 Disponibilidad | RecommendationService | Búsqueda | CA-04 |
| RF08 Recomendación | RecommendationService | Resultados | RecommendationTest, ClassroomRegistrationTest; CA-05 |
| RF09 Detalle y alternativas | RecommendationService, ClassroomController | Detalle y alternativas | CA-05 |
| RF10 Reservas | ReservationController | Confirmar y cancelar | ReservationConflictTest; CA-06, CA-07 |
| RF11 Calendario | ReservationController | Lista y calendario semanal | CA-07 |
| RF12 Avisos y reportes | ReservationAlertService, NotificationController, ReportController | Inicio, alertas emergentes, notificaciones y reportes | ReservationAlertTest, ui_behavior_test; CA-11, CA-12 |

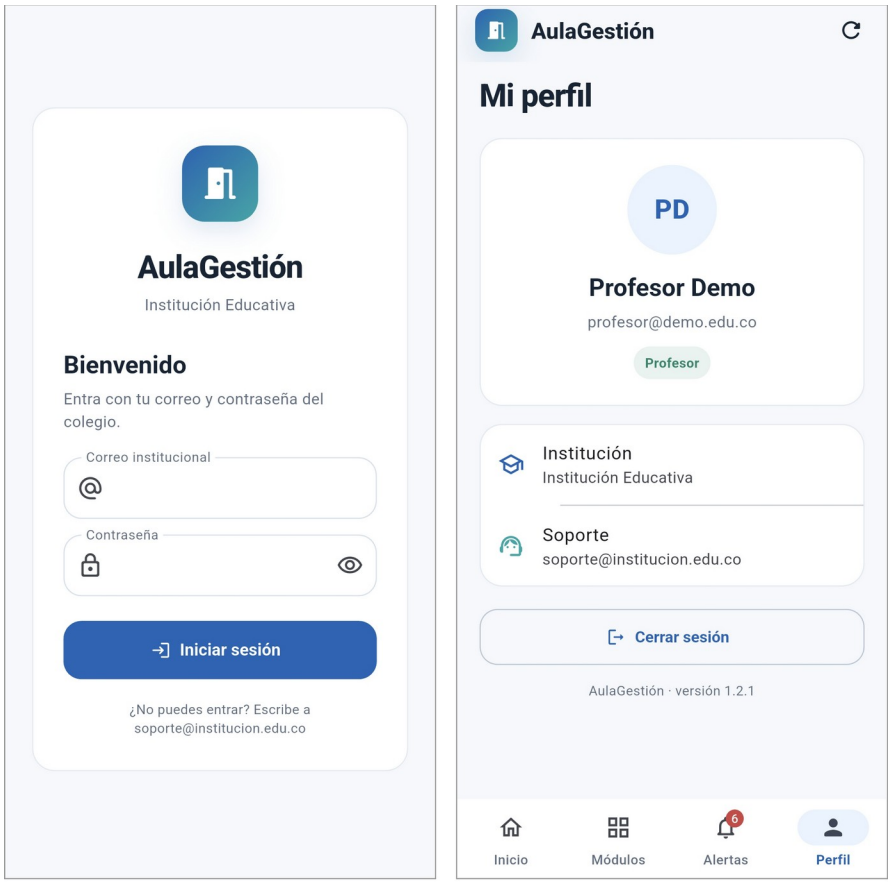
# Apéndice D

## Evidencias de funcionamiento

Las siguientes capturas se tomaron el 8 de noviembre de 2026 en un teléfono Android con la versión 1.2.1 instalada y conectada a la API pública. Se usaron las cuentas de demostración.

**Figura D1**

*Inicio de sesión y perfil*



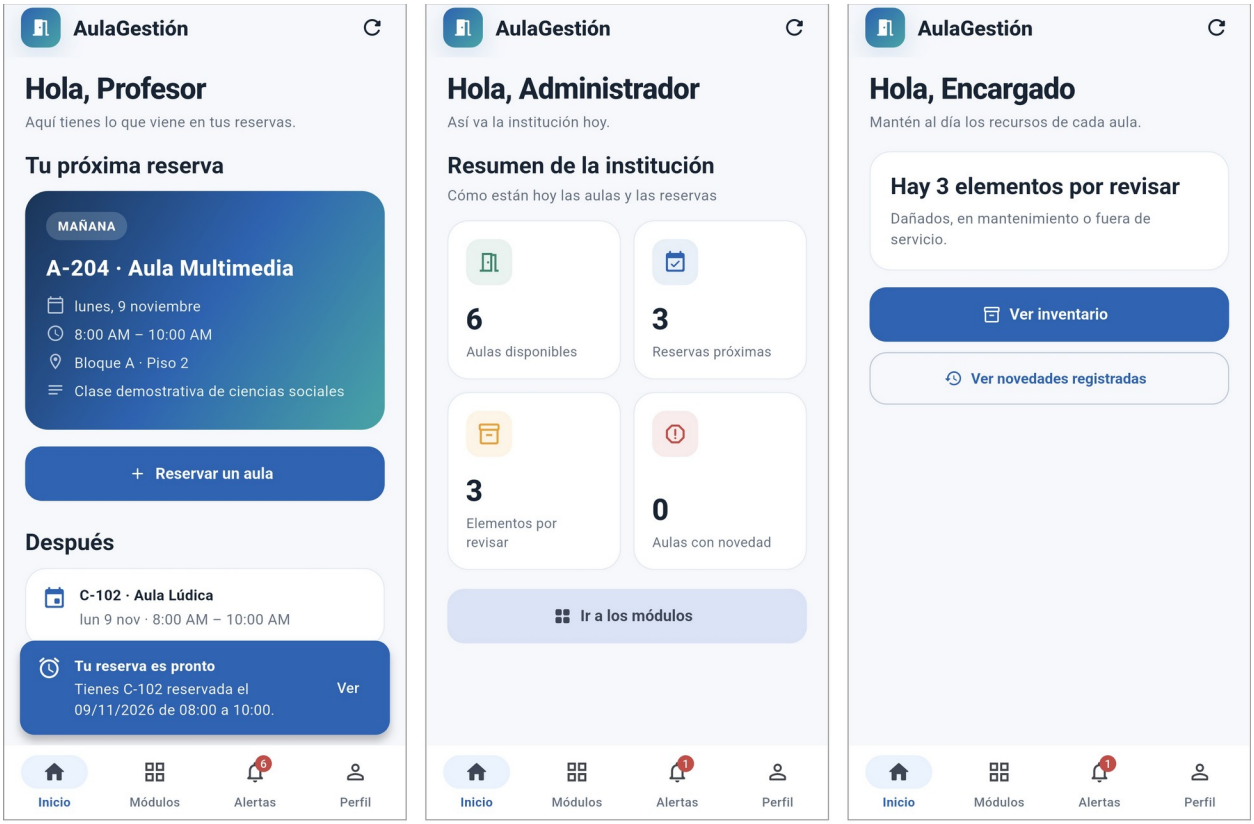

a. Inicio de sesión

b. Perfil del profesor

*Nota.* El perfil muestra el rol de la cuenta y la versión instalada de la aplicación.

**Figura D2**

*Pantalla de inicio según el rol*

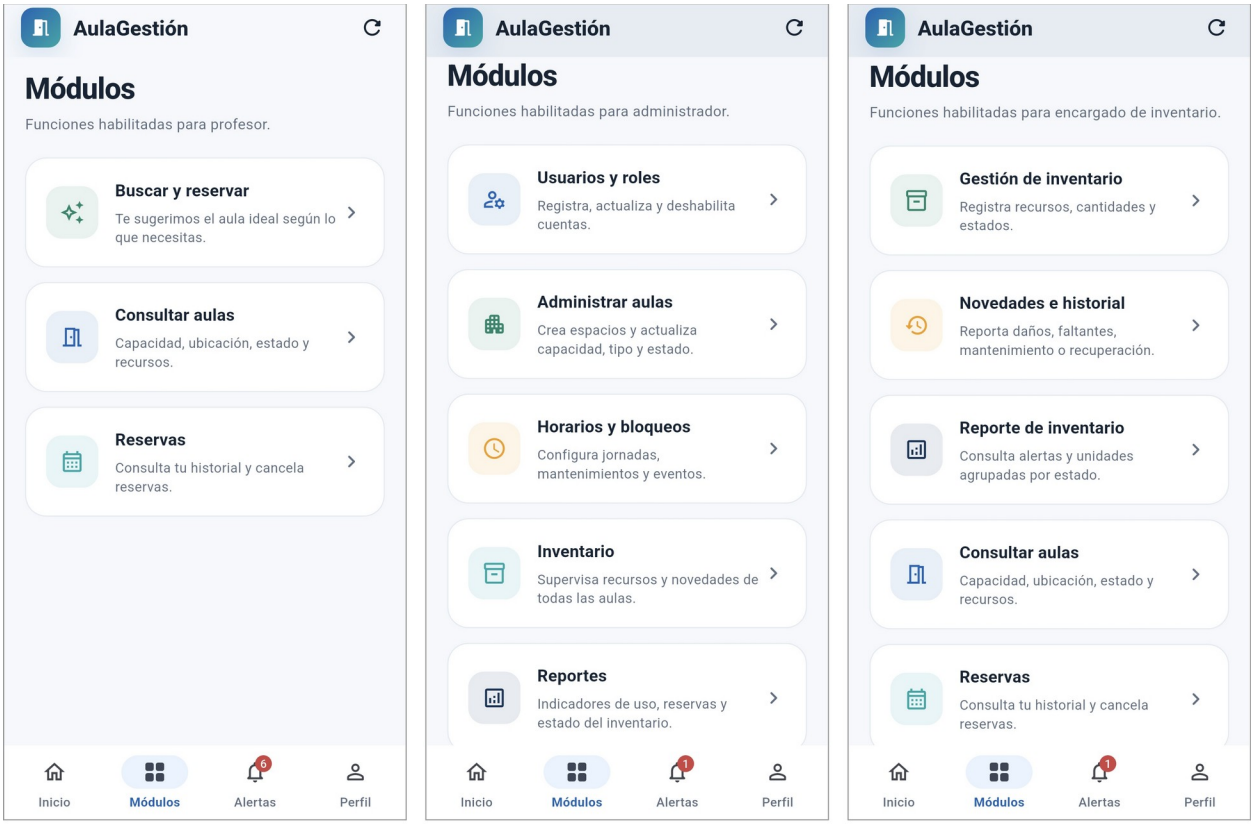



a. Profesor b. Administrador c. Encargado de inventario

*Nota.* El profesor ve primero su próxima reserva y, abajo, un aviso breve de recordatorio; el administrador ve el resumen de la institución; el encargado de inventario ve los elementos por revisar.

**Figura D3**

*Módulos habilitados para cada rol*

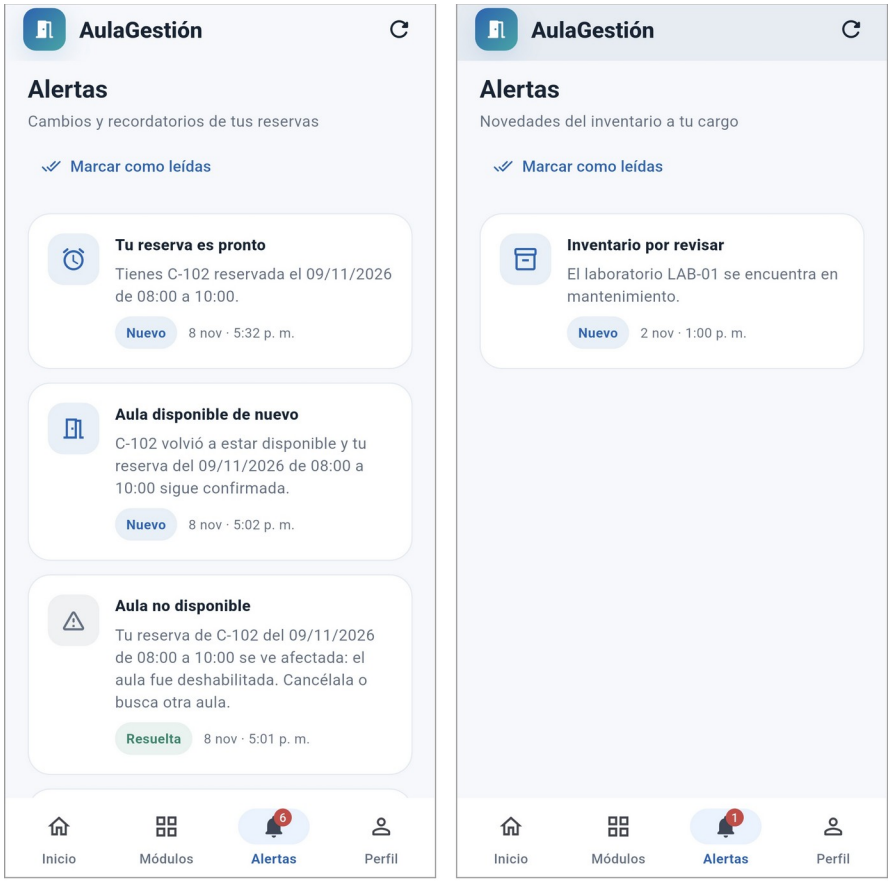



a. Profesor b. Administrador c. Encargado de inventario

*Nota.* El menú se construye según los permisos de la Tabla 4. La lista del administrador continúa con inventario, reportes, recomendador, aulas y reservas.

**Figura D4**

*Historial de alertas y avisos*




a. Profesor

b. Encargado de inventario

*Nota.* En a se ve el ciclo completo de la Figura 5: la alerta de aula no disponible quedó resuelta, llegó el aviso de aula disponible de nuevo y luego el recordatorio de la reserva. En b, la novedad de inventario que recibe el encargado.